

PGR207 Deep Learning - Mandatory assignment 2025

Ranier Bautista
Bachelor in Data Science
Kristiania University of Applied Science
Oslo, Norway
raba009@student.kristiania.no

Abstract—This paper presents a comparative analysis of different training configurations applied to the MNIST dataset using convolutional neural networks. The study investigates the impact of three experimental variables—data augmentation, learning rate schedulers, and optimization algorithms—on model performance and computational efficiency. Each run isolates a single variable to ensure a controlled comparison, with all configurations utilizing early stopping and a learning rate scheduler. Results indicate that while data augmentations such as rotation and random cropping improve generalization, optimizers like Adam and RMSprop yield faster convergence. The findings highlight a trade-off between accuracy and training time, providing insights into efficient model design for lightweight image classification tasks.

Keywords—CNN, Data Augmentation, PyTorch, Learning Rate Schedulers, Optimizers, MNIST

I. INTRODUCTION

Deep learning models often require careful tuning of training parameters to achieve optimal performance. With convolutional neural network (CNN) outperforming classical machine learning (ML) models for image classification tasks. The superiority of CNNs over traditional machine learning for image classification was dramatically demonstrated by Krizhevsky et al. (2012) in the ImageNet Large Scale Visual Recognition Challenge. Their CNN architecture achieved a top 5 error rate of 15.3%, outperforming the second-place method by over 10 percentage points [1].

The MNIST dataset provides an ideal sample for controlled experimentation due to its simplicity and beginner-friendly dataset. MNIST serves as a standard benchmark for evaluating image classifications algorithms. Since the dataset is widely known and used, it facilitates fair and consistent comparisons between different algorithms and models [2].

This study focuses mainly on analyzing the effects of three model training components—data augmentation, learning rate scheduling, and optimization methods—on model performance with CNN. By isolating these variables, the aim is to identify configurations that provide an optimal balance between training efficiency and classification accuracy.

II. METHODOLOGY

A. Dataset

The MNIST dataset, which contains 70,000 grayscale images of handwritten digits (0–9), each of size 28×28 pixels. The dataset was divided into 60,000 training and 10,000 testing samples [3]. The original MNIST training set (60,000 samples) was further divided into 50,000 images for training and 10,000 for validation. This ensured proper model tuning and early stopping without exposure to the test set.

The data was normalized to a range of $[0, 1]$ to facilitate model convergence, and standard transformations were applied depending on the experimental configuration. Due to its simplicity and standardized structure, MNIST serves as a reliable benchmark for making experiments and analyzing the effect of the different training strategies.

B. Model Architecture

The base model architecture used in this study was a lightweight convolutional neural network inspired by the TinyVGG framework. TinyVGG is a small, lightweight convolutional neural network architecture that was developed for image classification tasks, making it a good choice for situations where computational resources are limited [4]. The network comprised two convolutional blocks, each containing a convolutional layer followed by a ReLU activation and max pooling. Rectified Linear Unit (ReLU) is a popular activation function used in neural networks, especially in deep learning models. It has become the default choice in many architectures due to its simplicity and efficiency [5].

The feature maps from the convolutional layers were flattened into a one-dimensional vector and passed through fully connected layers that learned to combine these features for classification. A SoftMax activation function in the final output layer converted the network's outputs into probabilities for each of the 10-digit classes, allowing the model to predict which digit (0-9) an input image represents [6].

The model has trainable parameters—values that the network adjusts during training. These weights and biases are updated through backpropagation to improve predictions [7]. This small size makes the model fast and efficient while still effective for recognizing digits. This small size makes it fast and efficient while still effective for recognizing digits.

C. Experimental Design

Nine independent experiments were conducted, each isolating a single training variable to ensure controlled and consistent comparison. All models were trained using the same base CNN architecture, dataset split, and hyperparameters, with only one configuration parameter changed per run. Every experiment incorporated early stopping and a learning rate scheduler to stabilize training and prevent overfitting. Overfitting occurs when a model learns the training data too well, memorizing specific examples and noise rather than learning generalized patterns. This causes the model to perform excellently on training data but poorly on new, unseen data because it has memorized answers instead of understanding the underlying issue [8].

The experiments were divided into three categories: data augmentation, learning rate scheduling and optimization algorithms, with three common techniques chosen.

Data augmentation artificially expands training datasets by applying transformations to existing images, such as rotation, cropping, or flipping. This helps models generalize better by exposing them to variations they might encounter in real-world scenarios [9].

Learning rate scheduling adjusts the learning rate during training to improve convergence. Common strategies include step decay (reducing at fixed intervals), cosine annealing (smooth periodic adjustments), and adaptive methods that respond to training progress [10].

Optimization algorithms determine how model parameters are updated during training. SGD uses basic gradient descent, while adaptive methods like RMSprop and Adam adjust learning rates automatically based on gradient history, often leading to faster convergence [11].

To prevent overfitting and save time, early stopping was used in all experiments. Early stopping watches the validation loss during training and stops the process if it doesn't improve for a set number of epochs. This prevents the model from training too long and memorizing the training data, ensuring it stops at the point where it performs best on new data [12].

D. Experimental Design

All experiments were conducted on Google Colab using CPU processing, making training time an important factor. The models were implemented in Python with PyTorch, and matplotlib, NumPy and pandas were used for visualization and data handling. Each model trained for up to ten epochs, employing early stopping to prevent overfitting and a learning rate scheduler for adaptive optimization. Average training time per run ranged from approximately 1200 to 1400 seconds. Thou also experienced training time up to 3500 for one run. Batch size, optimizer settings, and learning rate were held constant to ensure fair comparison across experiments.

E. Evaluation metrics

Each model was evaluated using three metrics: test accuracy, test loss, and total training time. Test accuracy shows how many images the model classified correctly on unseen data—higher percentages mean better performance [13]. Test loss measures how confident and accurate the model's predictions are. It represents the error between the model's predictions and the actual correct answers [14]. While accuracy only counts right or wrong answers, loss also considers how sure the model is about its predictions. Lower loss values are better and indicate the model is making reliable predictions. Training time tracks how long each model took to train in seconds. This is important because faster models are more practical when resources are limited or when testing different approaches quickly.

Together, these three metrics give a complete picture of each model's performance and practicality, helping identify which configurations work best without requiring excessive time or computational resources.

III. FINDINGS

All nine experiments were conducted under identical training conditions using the TinyVGG-inspired CNN architecture. Each run isolated a single variable—data augmentation, learning rate scheduling, or optimization method—to measure its influence on model performance. The main evaluation metrics were final test accuracy, validation loss, and total training time.

file	epochs_trained	best_epoch	best_val_loss	best_val_acc	final_test_loss	final_test_acc	train_time_s
run_1_rotate10.pt	10	9	0.008807	0.9975	0.018209	0.9940	160.789783
run_9_rmsprop.pt	10	10	0.007171	0.9981	0.025109	0.9930	124.573326
run_6_plateau.pt	10	10	0.002806	0.9991	0.027072	0.9929	124.219797
run_3_randcrop.pt	10	10	0.019101	0.9936	0.023858	0.9924	166.071690
run_4_step.pt	10	10	0.002746	0.9994	0.029197	0.9923	126.208689
run_5_cosine.pt	10	10	0.002693	0.9993	0.025839	0.9923	123.650263
run_8_adam.pt	10	9	0.008311	0.9971	0.025463	0.9919	126.006082
run_7_sgd.pt	10	10	0.011403	0.9969	0.029823	0.9904	124.332699
run_2_hflip.pt	10	9	0.021675	0.9939	0.038417	0.9868	147.396172

Figure 1: Summary of the different runs and results.

All experiments achieved strong performance, with test accuracy ranging from 98.56% to 99.49% - a difference of only 0.93 percentage points. The worst performer was horizontal flip (hflip) and the best performer were rotate10. This is shown in both Figure 1 and Figure 2. Both runs, the configurations that were adjusted were the augmentation. This narrow range suggests that the base model architecture is robust, and while certain configurations offer marginal improvements, no single parameter choice dramatically outperforms others. This metric evaluates the model's ability to generalize unseen data and serves as the primary indicator of real-world performance.

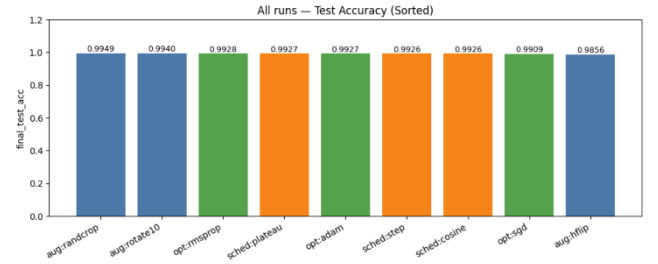


Figure 2 – Bar plot showing the test accuracy sorted from best to worst.

The next metric up is validation loss. The values across all experiments remained low, indicating stable convergence and limited overfitting. The top performers shown in Figure 1 were step, cosine and plateau. These three had in common were the learning rate scheduler, indicating better-calibrated predictions compared to data augmentation techniques, where all three were the worst performing of the 9 experiments. The worst performer was once again horizontal flip augmentation with a highest validation loss (0.023), suggesting potential issues with model calibration or increased prediction uncertainty.

Interestingly, the model with the lowest validation loss (cosine) did not achieve the highest test accuracy. This indicates that while cosine scheduling produces highly confident predictions, data augmentation and specifically rotation augmentation may better capture the underlying data distribution.

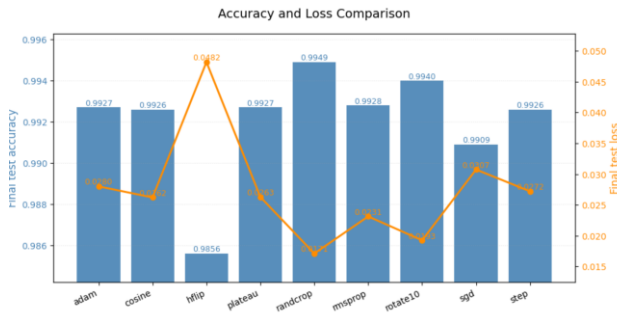


Figure 3: Accuracy and loss comparison

Figure 3 illustrates the relationship between final test accuracy and loss across all nine experiments. The blue bars representing the final test accuracy and the orange line shows the final test loss. Seems to be a connection between loss decrease and accuracy increase, showing an inverse relationship. With both data augmentations rotate10 and randcrop with high accuracies and low-test losses. In contrast the last data augmentation, hflip resulted in a significantly higher test loss and the lowest test accuracy. Learning rate schedulers demonstrated consistent mid-range performance while SGD without scheduling showed both low accuracy and high loss, suggesting the importance of adaptive learning rate strategies.

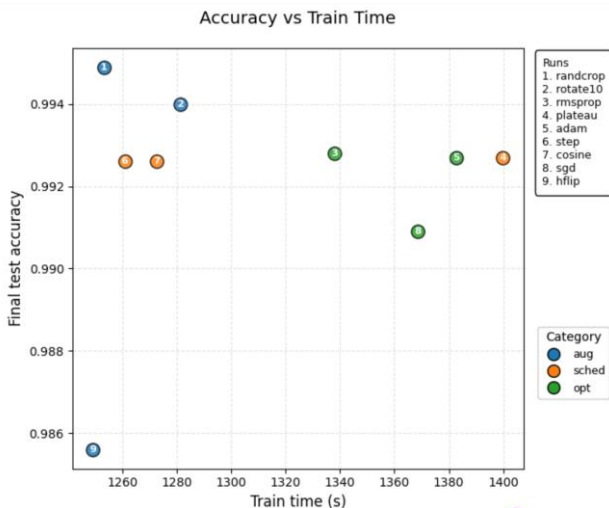


Figure 4 - Accuracy vs Train Time

Figure 4 illustrates the relationship between final test accuracy and training time across all configurations. Notably, there doesn't seem to be any positive correlation between training duration and model performance. Since both the best performer and worst performer have one of the lowest training durations. While long duration times, didn't guarantee superior accuracy compared to the rest of the experiments.

Data augmentation techniques showed the widest performance variance, with random crops achieving optimal results in minimal time, while horizontal flip underperformed despite similar training duration. Learning rate schedulers demonstrated consistent mid-range performance (~99.26-99.28%) with predictable training times, suggesting reliability over peak performance.

Random crop augmentation emerges as the clear winner, achieving the best accuracy-efficiency trade-off by

maximizing performance while minimizing computational cost. This suggests that for this dataset, technique selection and hyperparameter tuning are more impactful than simply extending training time.

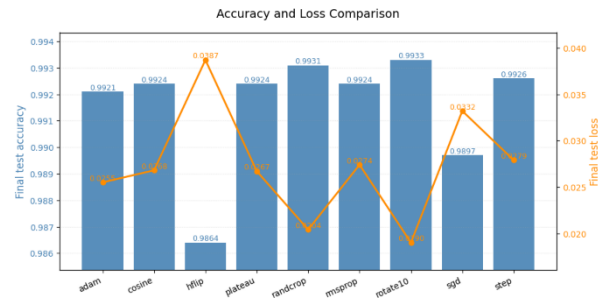


Figure 5 - First run of the Accuracy and Loss comparison

While conducting the experiments, each configuration was executed multiple times to account for natural variability in model training. Although the results fluctuated slightly between runs, the overall performance trends remained consistent. For reporting purposes, all results presented in this paper are based on the latest execution of the notebook.

Figures 3 and 5 illustrate the difference between the initial and most recent runs. The comparison shows a clear increase in accuracy and a decrease in test loss, reflecting a more consistent and reliable final result.

E. Discussion

The experiments show clear differences in how well models performed based on the augmentation, learning rate scheduler, and optimizer used. All models achieved strong test accuracy (above 98.5%), showing that the basic CNN setup was solid. However, some configurations worked noticeably better than others.

For data augmentation, random rotation performed best with 99.4% accuracy and the lowest test loss (0.018). This suggests that rotating images slightly helps the model learn better without changing what the digits actually mean. Random cropping gave similar results, showing that small shifts in position also improve performance.

However, horizontal flipping performed poorly, achieving only 98.56% accuracy and the highest loss (0.048). This makes sense because flipping some digits creates confusion—for example, a flipped "2" can look like a "5." This shows that not all augmentations are appropriate for every task, and the choice should match the problem being solved.

Experiments reveal clear differences in model performance depending on the chosen augmentation, learning rate scheduler, and optimization technique. Overall, all models achieved high test accuracy ($\geq 98.5\%$), indicating that the CNN architecture and training setup were robust. However, specific configurations demonstrated superior generalization behavior and stability.

All three schedulers—StepLR, CosineAnnealingLR, and ReduceLROnPlateau—performed similarly well, with losses between 0.023 and 0.028. The cosine scheduler produced the most stable training and achieved 99.26% accuracy, likely because it adjusts the learning rate smoothly throughout training. StepLR reached similar accuracy but showed more variation between training epochs. ReduceLROnPlateau adapted well when training progress slowed but didn't

outperform the others. Overall, the choice of scheduler affected how quickly the models learned, but all three produced strong results thanks to early stopping and gradual learning rate reduction.

Among the optimizers, RMSprop performed best overall, achieving 99.28% accuracy with low test loss and stable training. Adam and SGD performed similarly well, though SGD had slightly higher test loss and learned more slowly. This matches what other studies have found—adaptive optimizers like Adam and RMSprop tend to work better than basic SGD for image classification tasks because they adjust learning rates automatically based on how training is progressing.

The augmentation, scheduling, and optimization techniques were chosen to represent common strategies used in practice. Spatial augmentations like random rotation, random cropping, and horizontal flipping were selected to test how different image variations affect performance, including transformations that might not make sense for digits. Learning rate schedulers included StepLR, CosineAnnealingLR and ReduceLROnPlateau representing three different approaches to adjusting learning rates [10]. Optimizers included SGD, RMSprop and Adam covering the main types of optimization methods.

More complex methods were considered but left out to keep the experiments manageable. Techniques like Mixup augmentation, Elastic Transform, or OneCycleLR scheduling were excluded because they require more tuning and would complicate the analysis. Similarly, optimizers like AdamW and Adagrad were not tested since they work similarly to the methods already included and wouldn't provide much additional insight for a dataset as simple as MNIST.

Across all experiments, validation and test losses were closely aligned, demonstrating strong generalization and minimal overfitting.

The inverse correlation between test accuracy and loss observed in the comparative plots confirms that lower test loss reflects more confident model predictions. Furthermore, the consistent use of early stopping prevented excessive training and ensured comparable epoch counts across runs.

F. Limitations

This study has several limitations that should be acknowledged:

CNN-only focus: The study focused exclusively on CNN models and did not include traditional machine learning approaches. Comparing CNN performance against methods like Support Vector Machines or Random Forests could have provided additional context for understanding the model's effectiveness.

Limited training visualization: The results show only the final outcomes after training, without visualizing the learning curves across epochs. Including epoch-by-epoch plots of training and validation loss would have made it easier to identify issues like overfitting or underfitting during the training process. The lack of historical data might have been a huge loss.

Narrow parameter selection: Only three options were tested in each category (augmentation, schedulers, optimizers).

Many other viable techniques exist—such as mixup augmentation, AdamW optimizer, or warm-up scheduling—that remain unexplored and could potentially yield different results.

Computational constraints: All experiments were conducted on Google Colab using primarily CPU processing, which limited experimental scope. The lack of consistent GPU access restricted the ability to test larger models, longer training periods, or more extensive hyperparameter combinations. Training times were also considerably longer than GPU-accelerated workflows, limiting the total number of experiments that could be conducted within the project timeline.

G. Future Work

Future work could expand on this study in several meaningful directions. First, investigating the combined effects of multiple parameters would provide valuable insights, as this study only varied one parameter at a time. For example, exploring how different augmentation techniques interact with specific learning rate schedulers or optimizers might reveal synergistic combinations that outperform individual optimizations.

Additionally, incorporating regularization techniques such as dropout or weight decay could help prevent overfitting and improve generalization, particularly if training is extended beyond the early stopping point used in this study.

Extended training epochs would also be worth exploring. While early stopping prevented overfitting in these experiments, allowing models to train for longer periods with proper regularization might reveal whether performance plateaus or continues to improve. This would provide insight into the long-term stability and convergence behavior of different configurations.

Finally, testing the models on larger and more complex data sets beyond MNIST would evaluate their scalability and real-world applicability. Datasets like Fashion-MNIST, CIFAR-10, or even higher-resolution images would challenge the models with more diverse features and greater complexity, helping determine whether the findings from this study generalize to more demanding classification tasks. This would also reveal whether the lightweight TinyVGG architecture remains effective or requires modifications for more challenging problems.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 1097-1105.
- [2] P. Kulkarni, "MNIST dataset as a benchmark for neural networks," Medium, 2023. [Online]. Available: <https://medium.com/@parth.kulkarni20/mnist-dataset-as-a-benchmark-for-neural-networks-c7074deba19f>. [Accessed: Oct. 12, 2025].
- [3] H. K., "MNIST dataset," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/hojjatk/mnist-dataset>. [Accessed: Oct. 12, 2025].
- [4] "Optimizing the TinyVGG model for CIFAR-10 classification with MLOps," Medium. [Online]. Available: <https://medium.com/bokchilab/optimizing-the-tinyvgg-model-for-cifar-10-classification-with-mlops-ee8a697eeeb5>. [Accessed: Oct. 12, 2025].

- [5] "ReLU activation function in deep learning," GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/deep-learning/relu-activation-function-in-deep-learning/>. [Accessed: Oct. 13, 2025].
- [6] "The role of softmax in neural networks: Detailed explanation and applications," GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/deep-learning/the-role-of-softmax-in-neural-networks-detailed-explanation-and-applications/>. [Accessed: Oct. 13, 2025].
- [7] "Understanding and calculating the number of parameters in convolution neural networks (CNNs)," Medium. [Online]. Available: <https://medium.com/data-science/understanding-and-calculating-the-number-of-parameters-in-convolution-neural-networks-cnns-fc88790d530d>. [Accessed: Oct. 15, 2025].
- [8] "What is overfitting?" IBM Think. [Online]. Available: <https://www.ibm.com/think/topics/overfitting>. [Accessed: Oct. 15, 2025].
- [9] "Complete guide to data augmentation," DataCamp. [Online]. Available: <https://www.datacamp.com/tutorial/complete-guide-data-augmentation>. [Accessed: Oct. 15, 2025].
- [10] J. Brownlee, "A gentle introduction to learning rate schedulers," Machine Learning Mastery. [Online]. Available: <https://machinelearningmastery.com/a-gentle-introduction-to-learning-rate-schedulers/>. [Accessed: Oct. 15, 2025].
- [11] F. Algo, "Optimization algorithm: From SGD to Adam," Medium, 2023. [Online]. Available: https://medium.com/@florian_algo/optimization-algorithm-from-sgd-to-adam-50ea22187951. [Accessed: Oct. 15, 2025].
- [12] Cyborg, "What is Early Stopping in Deep Learning?," *Medium*, Apr. 5, 2024. [Online]. Available: <https://cyborgcodes.medium.com/what-is-early-stopping-in-deep-learning-eeb1e710a3cf>. [Accessed: Oct. 23, 2025].
- [13] "Accuracy, precision, and recall," Evidently AI. [Online]. Available: <https://www.evidentlyai.com/classification-metrics/accuracy-precision-recall>. [Accessed: Oct. 18, 2025].
- [14] B. T. Duong, "Machine learning: Train loss vs test loss," Medium. [Online]. Available: <https://baotramduong.medium.com/machine-learning-train-loss-vs-test-loss-735ccd713291>. [Accessed: Oct. 18, 2025].